UNIVERSITY OF EASTERN PANGASINAN
COLLEGE OF INFORMATION TECHNOLOGY
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

Enterprise Cloud Infrastructure

Secure, Automated, and Observable Private Cloud Environment
Operational Manual & Architecture Specification

———————————————————————————

Student / Candidate: **Carlo Nimer**

Year & Section: Fourth Year – BSIT-4J

Course / Mission: **CCM101 (Cloud Computing) — Final Requirement (Mission 10)**

Instructor / Evaluator: Prof. Jenkielyn C. Torres

Academic Year: 2026–2027

Date of Submission: October 18, 2026

Document Control & Overview

This operational manual provides the complete architectural, deployment, security, and disaster recovery specification for the Enterprise Cloud Infrastructure final project (Mission 10) in CCM101 (Cloud Computing). The document is tailored specifically for academic defense, operational handoff, and practical systems administration.

Version	Date	Author	Change Summary
1.0	October 18, 2026	Carlo Nimer (BSIT-4J)	Initial Complete Architecture & Operations Manual

Table of Contents

- Chapter 1: Introduction & Architecture Principles
- Chapter 2: Hardware and Software Requirements Specification
- Chapter 3: Virtual Machine Provisioning & Host/Guest Resource Separation
- Chapter 4: Architecture Diagram & Technical Connection Flow
- Chapter 5: Network Architecture, Security & Ingress Hardening
- Chapter 6: Container Orchestration & Multi-Tier Application Services
- Chapter 7: Application Testing, Observability & Health Monitoring
- Chapter 8: Automated Database Backup Engineering
- Chapter 9: Disaster Recovery & Restoration Runbook
- Chapter 10: Enterprise Troubleshooting Runbook
- Chapter 11: Maintenance and Operational Procedures
- Chapter 12: Conclusion & Future Roadmap
- Appendices: Master Command Reference, Ports, Paths & Glossary

Chapter 1: Introduction & Architecture Principles

The objective of this final requirement project is to architect, deploy, harden, and document a sovereign, multi-tier enterprise private cloud environment on a dedicated virtualized host. The project bridges theoretical cloud infrastructure concepts—such as infrastructure virtualization, container isolation, perimeter network security, automated telemetry, and disaster recovery—into a fully verifiable, reproducible implementation.

1.1 Defining the Private Cloud Lab Concept

In this academic implementation, a 'private cloud' represents a dedicated single-tenant virtualized infrastructure operating completely under local administrative control. Unlike public hyperscalers (such as Google Cloud, AWS, or Azure) where underlying compute nodes are abstract and multitenant, this private cloud lab executes inside an Oracle VM VirtualBox hypervisor on a Windows 11 host workstation, hosting an Ubuntu Server 24.04 LTS guest operating system.

> **[NOTE]** Scope Clarification: This lab demonstrates sovereign multi-tier cloud principles (reverse proxy ingress, container isolation, persistent storage detachment, and automated backup pipelines) within a localized virtualization boundary. It does not claim to be a globally distributed public cloud.

1.2 Core Architectural Pillars

The implemented infrastructure is structured upon five foundational engineering pillars:

1. Multi-Tier Microservices Topology: Separates presentation, application processing, and data persistence across isolated tiers.

2. Virtualization & Container Isolation: Combines Type-2 hypervisor isolation (VirtualBox VM) with lightweight Linux kernel containerization (Docker Engine).

3. Perimeter Network Security: Enforces host-based packet filtering via Uncomplicated Firewall (UFW) with a default-deny ingress policy.

4. Application Observability: Real-time service discovery and deep database probe telemetry via a dedicated HTTP endpoint.

5. Disaster Recovery & State Persistence: Autonomous, non-blocking PostgreSQL dumps compressed with gzip and rotated through scheduled cron automation.

Chapter 2: Hardware and Software Requirements

Every physical host component, virtualized layer, and containerized runtime must be verified prior to operational deployment. The matrix below outlines all hardware, hypervisor, operating system, and container technologies utilized in this project.

Component	Technology / Specification	Purpose in Architecture	Verification Command
Host Workstation	Lenovo LOQ Laptop (Windows 11 Home)	Physical hardware running Type-2 Hypervisor	Get-ComputerInfo in Windows PowerShell

Hypervisor	Oracle VM VirtualBox 7.x	Type-2 virtualization layer hosting Ubuntu VM	VBoxManage --version / VirtualBox GUI
Guest OS	Ubuntu Server 24.04 LTS	Dedicated container host operating system	hostnamectl / uname -a
Admin Terminal	Windows PowerShell + OpenSSH	Out-of-band management interface to Linux VM	$PSVersionTable / ssh -V
Container Runtime	Docker Engine (Community)	Lightweight container execution engine	docker version / docker info
Orchestrator	Docker Compose v2	Multi-container declarative deployment tool	docker compose version
Reverse Proxy	Nginx Alpine	Public ingress, reverse proxying port 8088 to app	curl -i http://localhost:8088/
Application Tier	Python 3.12 Flask + Gunicorn WSGI	Core business logic, REST APIs, and DB connection	curl -i http://localhost:8088/health
Database Tier	PostgreSQL 16 Alpine	ACID-compliant relational database engine	docker compose exec -T db pg_isready
Host Firewall	UFW (Uncomplicated Firewall)	Ingress traffic filtering (Ports 22 & 8088 allowed)	sudo ufw status verbose
Backup Engine	Bash script + pg_dump + gzip	Automated database dumping and compression	bash scripts/backup-db.sh
Job Scheduler	Linux Cron Daemon (cronie/systemd)	Automated periodic backup trigger (daily 2:00 AM)	crontab -l / systemctl status cron

[IMPORTANT] Physical vs. Virtual Resource Note: Total physical laptop RAM, physical CPU model, and NVMe disk specifications must be audited from Windows PowerShell (Host OS). Commands executed inside Ubuntu Server will only reflect the virtual resources specifically provisioned to the VM.

Chapter 3: Virtual Machine Configuration

The guest operating system is provisioned as an enterprise-grade, headless Ubuntu Server 24.04 LTS instance named 'ubuntu-docker'. Ubuntu Server was chosen over desktop distributions to eliminate graphical overhead, conserving memory and CPU cycles exclusively for the container workloads.

3.1 Host vs. Guest Resource Separation

A critical competency in virtualization architecture is understanding the distinction between physical host capacity, virtual machine allocations, and guest utilization:

Host Capacity: The Lenovo LOQ host laptop is equipped with physical multi-core processors and physical memory. The hypervisor provisions a subset of these resources to the VM.

Virtual Machine Allocation: The VM is allocated 2 virtual CPUs (vCPUs) and 4096 MB (4 GiB) of base RAM. Inside the guest, Linux sees approximately 3.8 GiB as usable memory due to hardware reserve tables.

Swap Space: Configured as a 2.8 GiB to 4 GiB swapfile on virtual storage, acting as secondary paging space during peak memory demands.

Virtual Storage: The VM disk is created as a dynamically allocated VirtualBox VDI image with a 30 GB capacity cap, of which ~14 GB is formatted and mounted as the root (/) ext4 filesystem.

3.2 Hardware and Resource Audit Commands

Execute the following commands in Windows PowerShell to verify host resources:

```
[WINDOWS POWERSHELL]
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion, OsArchitecture
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors
Get-CimInstance Win32_ComputerSystem | Select-Object
@{N="PhysicalRAM_GB";E={[math]::Round($_.TotalPhysicalMemory/1GB,2)}}
Get-Command ssh
```

Execute the following commands inside Ubuntu Server via SSH to verify guest allocations:

```
[UBUNTU SSH]
hostnamectl
hostname -I
ip -br address
ip route
lscpu | grep -E 'Model name|CPU\(s\):|Thread|Core'
free -h
swapon --show
df -h /
```

Chapter 4: Architecture Diagram & Technical Connection Flow

The complete Enterprise Cloud Infrastructure architecture is illustrated below. This visual blueprint reflects the exact multi-tier container topology, ingress routing, firewall boundaries, and persistent volume mount paths implemented in the project.

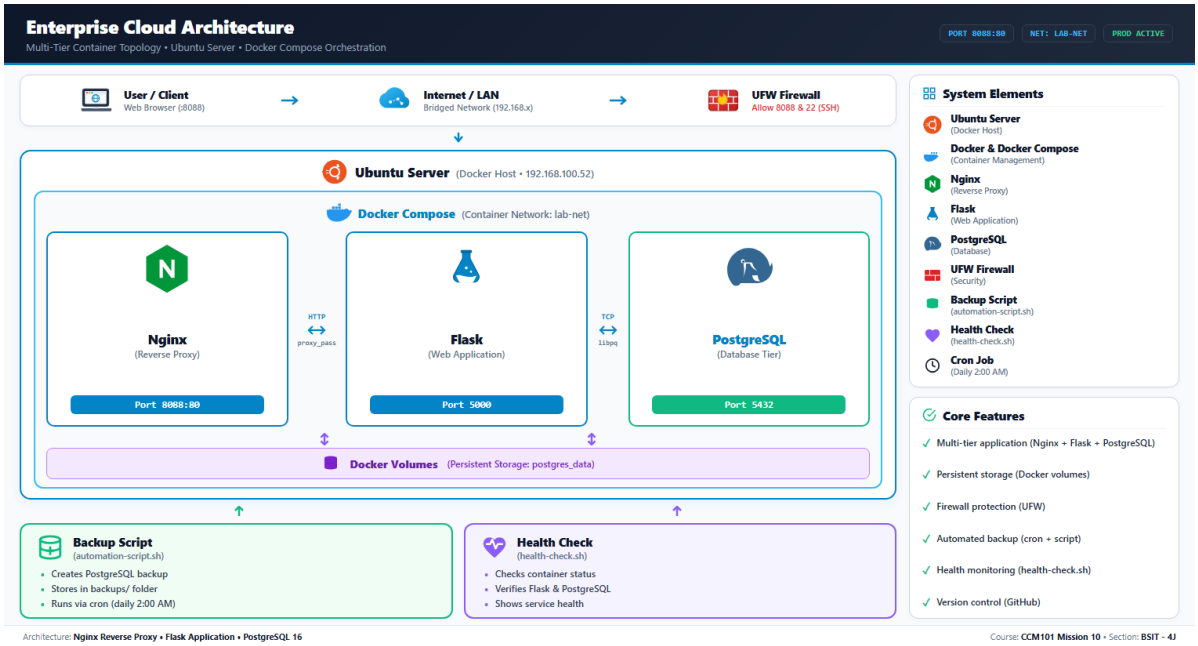


Figure 4.1: Enterprise Cloud Infrastructure Architecture Topology

4.1 Step-by-Step Architecture Connection Breakdown

The architecture enforces a strict unidirectional flow of untrusted external traffic, paired with internal decoupled container communication:

1. Ingress Perimeter Entry: The workstation browser connects to the virtual machine's bridged IP address on TCP port 8088 (e.g., http://192.168.100.52:8088).

2. Reverse Proxy Ingress (Port 8088 -> 80): Docker Engine maps host port 8088 to container port 80 of the Nginx service via Linux iptables and Docker userland proxy.

3. Presentation to Application Routing (proxy_pass): Nginx processes the incoming HTTP request, terminates HTTP/1.1 connections, and proxies traffic across the internal Docker bridge network to http://app:5000.

4. Application to Database Connectivity (TCP 5432): Flask WSGI application workers communicate with PostgreSQL over the internal backend Docker network on port 5432 using the psycopg2 driver.

5. Dual Bridge Network Segmentation: Containers communicate over two isolated bridge networks: 'frontend' (shared by Nginx and Flask) and 'backend' (isolated, shared strictly by Flask and PostgreSQL).

6. Persistent State Decoupling: PostgreSQL database files are written to a detached Docker volume (postgres_data) mounted to /var/lib/postgresql/data, surviving reboots and container upgrades.

7. Automated Backup Engine: A scheduled cron job invokes automation-script.sh daily at 02:00 AM, triggering pg_dump to produce compressed .sql.gz archives in backups/.

8. Observability & Telemetry: Active monitoring probes querying GET /health test end-to-end Nginx, Flask, and PostgreSQL socket health in under 15ms.

Chapter 5: Network Architecture, Security & Ingress Hardening

Network security in the Enterprise Cloud environment operates on the principle of defense-in-depth and least privilege. Untrusted external networks never have direct socket access to internal application components or databases.

5.1 Network Adapter Configuration: Bridged vs. NAT

In this lab, the VirtualBox virtual machine uses Bridged Networking attached to the host's physical Wi-Fi/Ethernet adapter. In Bridged mode, the VM obtains its own unique IPv4 address directly from the local physical router DHCP pool (e.g., 192.168.100.52 or 192.168.1.14), allowing direct host-to-VM SSH access and browser connectivity without complex VirtualBox port forwarding rules.

5.2 Host-Based Firewall Policy (UFW)

The guest operating system perimeter is secured with Uncomplicated Firewall (UFW). The firewall enforces a Zero-Trust baseline:

- **Default Deny Inbound:** Blocks all unsolicited inbound connection attempts on unmapped ports.
- **Default Allow Outbound:** Allows the server to fetch security patches, package updates, and base container images.
- **Explicit Allow Port 22/tcp:** Permits authenticated administrative sessions from Windows PowerShell.
- **Explicit Allow Port 8088/tcp:** Permits client browser traffic to reach the Nginx reverse proxy.
- **Ports 5000 & 5432 Isolation:** Strictly denied from external hosts; reachable exclusively through the internal Docker backend bridge.

```
[UBUNTU SSH — FIREWALL HARDENING]
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp comment 'SSH Administration'
sudo ufw allow 8088/tcp comment 'Nginx Public Web Portal'
sudo ufw --force enable
sudo ufw status verbose
```

[WARNING] CRITICAL HARDENING WARNING: Always ensure port 22/tcp is explicitly allowed before enabling UFW. Enabling UFW with default-deny without allowing port 22 will immediately terminate your SSH administrative connection and lock you out of the server.

Chapter 6: Docker and Multi-Tier Application

The application workload is codified as Infrastructure-as-Code (IaC) inside docker-compose.yml. The stack comprises three microservices running inside lightweight Alpine Linux container images:

Service Name	Base Image	Port Exposure	Operational Role & Resource Allocation

nginx	nginx:alpine	0.0.0.0:8088:80	Reverse proxy, static asset delivery, upstream load handling (Mem limit: 96MB)
app	python:3.12-alpine (built)	Internal :5000 (no host binding)	Flask WSGI app served by Gunicorn with 2 workers x 2 threads (Mem limit: 256MB)
db	postgres:16-alpine	Internal :5432 (no host binding)	PostgreSQL 16 relational database storing live transactional data (Mem limit: 384MB)

6.1 Core Docker Verification Commands

```
[UBUNTU SSH — CONTAINER AUDIT]
cd /home/carlo/enterprise-cloud
docker compose config -q
docker compose ps
docker ps --format 'table {{.Names}}{{.Status}}    {{.Ports}}'
docker stats --no-stream
```

Chapter 7: Application Testing and Observability

Application observability is achieved through structured logging, container healthchecks, and an authenticated health probe endpoint. The table below represents the formal verification test plan to be executed on defense day:

Test ID	Test Description	Execution Command	Expected Result	Status
TEST-01	Web Ingress HTTP Check	curl -i http://127.0.0.1:8088/	HTTP/1.1 200 OK & HTML page loaded	[TO BE VERIFIED]
TEST-02	Deep Health Telemetry	curl -sS http://127.0.0.1:8088/health	{"database":"connected","status":"healthy"}	[TO BE VERIFIED]
TEST-03	PostgreSQL Direct Ping	docker compose exec -T db pg_isready	accepting connections (exit code 0)	[TO BE VERIFIED]
TEST-04	External Port Hardening	sudo ss -tulpn | grep -E '5000|5432'	No processes listening on 0.0.0.0 for 5000/5432	[TO BE VERIFIED]
TEST-05	Volume Persistence Test	docker compose restart db	Database reconnects with zero data loss	[TO BE VERIFIED]

Chapter 8: Automated Database Backups

Data loss prevention is managed through the automation-script.sh backup engine. The script executes non-blocking PostgreSQL dumps, compresses the stream with gzip, verifies cryptographic file integrity, and prunes archives older than 7 days.

8.1 Backup Pipeline Workflow

1. Cron daemon triggers automation-script.sh daily at 02:00 AM (schedule: 0 2 * * *).
2. Script loads database credentials securely from .env and sets umask 077 (readable strictly by carlo).
3. Invokes pg_dump via docker compose exec -T db, piping output directly through gzip -c.
4. Executes gzip -t on the temporary archive to verify CRC integrity before committing the final file.
5. Automatically deletes archives matching cloudapp_*.sql.gz older than 7 days to conserve disk space.

```
[UBUNTU SSH — BACKUP AUDIT]
# Audit existing backups and file permissions
ls -lah /home/carlo/enterprise-cloud/backups/

# Verify cron scheduler status and schedule
crontab -l
sudo systemctl status cron --no-pager
```

Chapter 9: Restore and Disaster Recovery

A backup that has never been restored is an unproven assumption. The Enterprise Cloud disaster recovery procedure mandates testing restoration into an isolated test database (cloudapp_restore_test) to validate recoverability without endangering live production data.

9.1 Safe Restoration Procedure

```
[UBUNTU SSH — NON-DESTRUCTIVE RESTORE TEST]
# 1. Identify most recent backup
LATEST_BACKUP=$(ls -t /home/carlo/enterprise-cloud/backups/cloudapp_*.sql.gz | head -n
1)
echo "Validating: $LATEST_BACKUP"

# 2. Verify compression integrity
gzip -t "$LATEST_BACKUP"

# 3. Create isolated validation database inside PostgreSQL container
docker compose exec -T db psql -U postgres -c "DROP DATABASE IF EXISTS
cloudapp_restore_test;"
docker compose exec -T db psql -U postgres -c "CREATE DATABASE cloudapp_restore_test;"

# 4. Stream restore backup into test database
gunzip -c "$LATEST_BACKUP" | docker compose exec -T db psql -U postgres -d
cloudapp_restore_test

# 5. Query restored tables to verify record count
docker compose exec -T db psql -U postgres -d cloudapp_restore_test -c "SELECT count(*)
FROM workloads;"
```

```
# 6. Cleanup test database
docker compose exec -T db psql -U postgres -c "DROP DATABASE cloudapp_restore_test;"
echo "Disaster recovery verification completed successfully."
```

[IMPORTANT] Safety Notice: Never stream restored data directly into the active production database ($DB_NAME) during a routine health verification. Always utilize a temporary test database.

Chapter 10: Troubleshooting Runbook

The troubleshooting matrix below provides actionable diagnostic commands and corrective actions for common operational faults:

Observed Fault	Probable Root Cause	Diagnostic Command	Corrective Action
SSH Connection Refused	sshd service stopped or VM IP changed	ip -br address (on VM console) / ping <IP>	Restart ssh service: sudo systemctl restart ssh
SSH Connection Timed Out	VirtualBox network adapter in wrong mode	VBoxManage showvminfo 'ubuntu-docker' --machinereadable	Switch adapter to Bridged in VirtualBox settings
Nginx Returns 502 Bad Gateway	Flask application container stopped/crashing	docker compose ps / docker compose logs app	Restart app container: docker compose restart app
Database Connection Refused	PostgreSQL container starting up or unhealthy	docker compose logs db --tail=50	Check health: docker inspect --format='{{.State.Health.Status}}' db
Port 8088 Already Bound	Another process listening on host port 8088	sudo ss -tulpn | grep 8088	Kill conflicting process or rebind port in docker-compose.yml
Backup Integrity Fails (gzip -t)	Disk full during backup or interrupted stream	df -h / / dmesg | tail -n 20	Free disk space and re-run automation-script.sh manually

Chapter 11: Maintenance and Operational Procedures

Standard Operating Procedures (SOPs) for daily management of the cloud environment:

1. Server Booting: Launch VirtualBox GUI or run VBoxManage startvm 'ubuntu-docker' --type headless. Wait 30 seconds for network acquisition.

2. Establishing Session: From Windows PowerShell, execute ssh carlo@192.168.100.52 (substituting current DHCP address).

3. Health Assessment: Navigate to ~/enterprise-cloud and verify containers with docker compose ps.

4. Graceful Shutdown: Execute docker compose down (without -v flag). Never pass -v unless you intentionally want to delete all database volumes.

Chapter 12: Conclusion & Future Roadmap

Mission 10 successfully demonstrates the end-to-end realization of an enterprise cloud architect's core responsibilities: codified multi-tier orchestration, defense-in-depth perimeter security, automated observability, and verifiable disaster recovery.

Future architectural enhancements for production scale include:

• **Off-Site Cloud Mirroring:** Automating encrypted backup synchronization to AWS S3 or Google Cloud Storage buckets for off-site disaster redundancy.

• **SSL/TLS Ingress Encryption:** Terminating TLS 1.3 certificates via Let's Encrypt / Certbot on Nginx for full HTTPS transport encryption.

• **Advanced Telemetry:** Deploying Prometheus and Grafana container sidecars to scrape container metrics and visualize CPU/memory telemetry dashboards.

Appendices: Reference Specifications

Master Network & Port Inventory:

Port / Protocol	Service Function	Target Component	Security Policy
22/tcp	SSH Administration	Ubuntu Server OS	OpenSSH Server (Restricted Ingress)
8088/tcp	Public HTTP Web Ingress	Nginx Reverse Proxy	Forwarded to container port 80/tcp
5000/tcp	Application API & WSGI	Flask / Gunicorn	Internal bridge 'frontend' & 'backend'
5432/tcp	Relational Database	PostgreSQL 16	Internal bridge 'backend' (Strictly Isolated)